



OPENORMUS

A studio for fiction characters, and the *dialogues* between them.

Define characters with real psychology. Let an LLM play them in a scene.
Then measure whether it kept them in character.

BUILD · DIRECT · MEASURE

You wrote three distinct characters.

A hero with a sharp tongue. A cynical mentor.
A sidekick who never says what they mean.
You ask an LLM to write the next scene.

*And all three sound
like the same polite assistant.*

OUTPUT · ONE DENSE PROMPT

HERO

"I'd be happy to help with that. Let's work through it together."

MENTOR

"Great question! Here's how I'd approach it, step by step."

SIDEKICK

"Absolutely, I'm here for whatever you need."

● collapsed to the average

A character isn't a prompt. It's a *structured sheet*.

The more atomic and separated the information, the more reliably a model holds a character across a long scene. Separate fields must each be satisfied. A fused paragraph just gets averaged away.

01 Identity - name - short description	02 Personality - traits - values - fears - goals - coping style	03 Voice - speech patterns - notable quotes	04 History backstory	05 Network relationships
---	--	--	---	---

The system prompt is assembled from the sheet, never written by hand.

Two paths into the same database.

PATH A

best for original characters

Manual · the wizard

[Basics](#) → [Personality](#) → [Connections](#)

A guided builder, one field at a time. Best when you already know exactly who the character is.

PATH B

research pipeline

Research · type a name

- 1 Exa search → franchise + main character list
- 2 Per character, in parallel: BASICS + DETAILS calls (Exa caps each call at 10 fields)
- 3 Merge → Zod validation → prefilled wizard

EVERY LLM RESPONSE =
UNTRUSTED INPUT

A Zod schema describes the exact shape we expect. Wrong type, missing field, malformed list: the call fails. The user never sees a half-broken character.

A conversation is not a *chat*.

Each turn is a single LLM call that returns three layers, not just a line of dialogue.

private Thought and emotion stay inside the character. Only you, the director, see them.

visible Everyone in the scene hears the spoken line. The action lives inside it, in italics.

ONE TURN · WALTER WHITE

single LLM call

◆ THOUGHT · PRIVATE REASONING

"He's stalling. He never believed in me."

◆ EMOTION · PRIVATE **anger · high**

subtext · shame disguised as aggression

→ SPOKEN LINE · EVERYONE IN THE SCENE SEES THIS

"I know. I know." *looks away, picks up a tool*

actions are written inline, as italics

Who speaks next?

STRATEGY A

Round robin



Equal turns, predictable order. Good for structured back-and-forth.

STRATEGY B

The *orchestrator*

A separate LLM call, before every turn.

IT READS

story + full conversation
(text + gestures) + participant
names

IT NEVER SEES

any character's psychology. Only
what has been said and done

It decides who should speak next, and why.

The moment nobody *programmed*.

A tense argument. One character is volatile
— the type who walks out when cornered.

no rule We never told the orchestrator. There is no "leave the scene" feature.

what happened After losing the argument publicly, they wrote — in their own dialogue — that they were done, and left.

Two separate models. No coordination. The same conclusion.

IN-CHARACTER DIALOGUE

"I'm done here. I'm leaving."

↓ the orchestrator read this

ORCHESTRATOR

stopped assigning them turns for the rest of the scene.

Every scene is anchored to a *story*.

And you're not watching from outside the glass. You can step into the scene as a character and take a full turn yourself.

STORY IMPROVER • LLM PASS

rough draft → sensory detail → sharper stakes → scene opens

Participants are introduced by name: characters, and you. No one sees anyone else's psychology; only what is said and done in the room.

ON EACH TURN

A cast character

thought • emotion • text • gesture

You • in a role

same shape

thought • emotion • text • gesture

Take a character's turn yourself — or just drop a line, or skip. You choose how far in you step.

Talk to the app. Get the *result*, not a receipt.

Characters, scenes, the library: everything is reachable through a natural-language agent. You say what you want; it decides which tools to call.

The interface isn't generic. When the agent creates a character, you don't get a confirmation string. You get the character card.

You "Import Walter White from Breaking Bad."

AGENT • TOOL CALLS

show_search ● done

character_search ● done

character_create ● done



Walter White

proud

calculating

ready to use

One tool registry. The browser, the agent, *any* client.

All nine tools are exposed over MCP, the open Model Context Protocol, so any compatible AI client can drive OpenOrmUS.

AUTHENTICATION · OAUTH2

External client → browser login → full access

no token to copy

no config to manage

your existing account

THE CLAUDE PLUGIN · THREE PARTS

AGENTS two: openormus and scene-director

COMMANDS 9 slash commands

/openormus:create-character

/start-conversation

/evaluate-conversation

HOOKS automatic behaviours, defined in hooks.json

To measure fidelity, we invented *Vethara*.

A fictional island city-state with zero footprint in any training data. Use Walter White and you test the model's memory, not its fidelity.

Tier 1 · 8 archetypes

A 2D grid: moral axis × agency axis. Maximally separated in personality space.

Idealist

Cynic

Empath

Pragmatist

× high agency

× low agency

Tier 2 · 4 twin pairs

the real test

Within each pair, every field is identical except one. A model that collapses to average cannot tell them apart. No partial credit.

Officials

speech patterns

Survivors

coping style

Reformers

fears

Caregivers

goals

32 scenarios: 8 social contexts × 4 pressure types. 18 of 24 were revised after a purity check: pressure that doesn't create genuine tension isn't a test.

Four passes. Four questions.

PASS 1

generate

Same code the user runs.

Production `generateTurn()` for each character × scenario. No scoring yet.

PASS 2

judge

The guessing game.

Judges match aliased characters from behaviour alone. Deterministic shuffle, seeded by scenario.

PASS 3

reconstruct

Does the profile survive?

Infer the profile blind, score against ground truth. Metric: **contradiction rate, not recall.**

PASS 4

drift

Did the pressure hold?

Scenario-aware. Was the scenario felt, and answered consistently across time segments?

HIGH fidelity • LOW engagement

consistent, but the scenario didn't land: a design issue.

You can identify *what* failed, not just that something did.

LOW fidelity • HIGH engagement

the scenario worked, but the model broke: a prompting issue.

Built by *two*.



Leonardo Berselli

CONVERSATION ENGINE · EVALUATION HARNESS

- › Turn strategies: round robin & orchestrator
- › Private reasoning & emotion output
- › Background-job runner & character form wizard
- › Agent chat UI, conversation views & design system
- › Evaluation pipeline (4 passes), dashboard & token tracking



Davide Andreolli

INFRASTRUCTURE · AGENT · DATASET

- › Auth, MCP server & OAuth (JWT)
- › Character research: Exa, CRUD, images, fuzzy search
- › AI agent loop & conversation tools
- › Evaluation dataset: 16 characters, 8×4 matrix, purity check
- › The Claude plugin: agents, 9 commands & hooks

SHARED CORE

One monorepo, one shared tool registry: the contract behind all nine tools, used by the browser, the agent, and MCP alike.



Characters that stay *themselves*.

Build them. Direct the scene. Measure the fidelity. Thank you. Happy to go deep on any layer.
